Memglide: Auditable, Bounded Repository Context for Coding Agents — Supplementary Material

SREE CHARAN REDDY KAILASAM

This supplement records the twelve prospective task contracts, pinned public inputs, original public-check counts, allocation and measurement protocol, source revisions, and reproducibility boundaries for the installed Memglide ablation. It also retains twenty-three historical study stages and all thirty adaptive development rounds, including rejected candidates and failed attempts. Native coding outcomes are reported in the accompanying paper; historical cohorts and local controls are kept separate.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Information systems** → *Information retrieval*.

Additional Key Words and Phrases: coding agents, repository context, source retrieval, reproducibility, software engineering

1 Scope and reading guide

The accompanying paper studies repository-context preparation around the installed Codex coding agent. This supplement makes the task contracts and measurement lineage readable without treating every development observation as independent confirmation. Its public-library feature tasks differ from the historical GitHub issues evaluated by SWE-bench [1]. The underlying model weights were not trained or changed for this work.

Identifiers p01–p06 refer to the Python lane; t01–t06 refer to the JavaScript/TypeScript lane. Repository IDs pr01–pr06 and r07–r12 are local provenance identifiers, not experimental outcomes. "Native session" means a coding-agent evaluation through `codex exec`. Source preparation, SDK calls, zero-model handoffs, local component tests and `codex sandbox` grader processes are different forms of evidence. Their counts must not be added to native session counts. Internal provider inference counts and account billing are unavailable.

All monetary figures below are observed-token tariff equivalents under the stated accounting convention. Missing usage remains unknown. A failed attempt retains its cost when usage is known. Reused-task repairs, post-hoc subsets and alternate tariffs remain labeled. The history inventory records 66 matching report/research files, including two archived duplicates; this inventory is a report review rather than an independent replay of every historical stream.

2 Prospective task panel

2.1 Selection and preserved inputs

The panel contains one author-designed extension in each of twelve small, pinned public repositories. Six are Python projects. Two JavaScript/TypeScript tasks require JavaScript changes only; four require runtime and declaration changes. Four roots share the maintainer Sindre Sorhus. This purposive panel is not a representative sample of software maintenance, and repository novelty is relative to the earlier recorded Memglide roots rather than the model's unknown training data.

The original archive, retained-file hashes, public source, tests, configurations, dependency manifests, lockfiles and license notices were kept. The Python provenance records no excluded files. Full file-level manifests in the public provenance documents are the authority for exact bytes. A public baseline count measures existing behavior; it is not a count of feature requirements or proof of complete compatibility. Across the twelve roots, original behavioral baselines

Author's Contact Information: Sree Charan Reddy Kailasam, srcharan014@gmail.com.

contain 443 passing checks, with 14 intrinsic platform skips in Colorama. Four roots additionally run their original TypeScript declaration assertions.

Two candidates were rejected before model outcomes. The pinned jdunck/python-unicodecsv candidate at cedbcc7f9642e6040b1a579152baf0bfce9a815a depended on unchanged unittest2 1.1.0 tests that could not import on Python 3.12 because collections.MutableMapping had been removed. Its obsolete dependency target, including an argparse package that could shadow the standard library, was isolated from the final profile. Colorama was the predeclared compatibility backup. The initial `lukeed/totalist` candidate at 4b071d3e54d466a4059de89006c752da782c8b02 had an upstream ESM test setup incompatible with the common Node 24 runtime and was replaced before outcomes. The original fixtures and screening failures remain in the authoring record. Neither rejection was based on a coding-model result.

Public readiness also required environment corrections. TERM=xterm is shared by every arm and grader. Termcolor's 89 functional checks and 10 benchmark smoke checks run in separate processes to avoid an upstream color-detection cache interaction; all 99 assertions remain. Benchmark assertions run with --benchmark-disable, so these are behavioral smoke checks rather than performance measurements. AVA uses the supported external `{cache: false}` configuration. Exact test-name/count parity with original discovery was checked; source, public tests and repository configuration were unchanged. Initial unsupported-cache failures and a separately recorded pre-execution sandbox retry were retained. The escape-html quoteStyle values were clarified before freeze, with the prior wording preserved; task prompts were not tuned after readiness to force source delivery.

2.2 Public command conventions

The exact atomic argument vectors, environment updates, expected positive test counts and timeouts are in the frozen per-lane task/command JSON files. The short forms below are a readable transcription, not new commands substituted for those vectors. `{root}` denotes the fresh assignment repository.

Python commands use the pinned Python executable to run pytest with `-q -c /dev/null --rootdir {root} -p no:cacheprovider`. They set PYTHONDONTWRITEBYTECODE=1, PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 and an explicit PYTHONPATH containing the isolated extensions, `{root}/src` and `{root}`. Each Python atomic public command has a 45-second limit. Benchmark smoke commands also load pytest_benchmark.plugin explicitly and use --benchmark-disable.

The two Mocha commands use `--reporter dot test/`. AVA commands use `--tap --serial --config` with the frozen external no-cache configuration. Declaration commands use tsd with `--typings index.d.ts --files index.test-d.ts`; the strip-final-newline command explicitly selects its compatible tsd-legacy 0.31.0 entry point. Other declaration commands retain their frozen executable binding. JavaScript/TypeScript public commands have 60-second limits.

Every feature prompt below is reproduced verbatim. The common developer contract additionally requires production-only edits, relevant public and new regression checks, preservation of existing behavior, and exact protection of all existing tests, snapshots, configurations, manifests, lockfiles, documentation, license files and Git metadata. New regression-test files are allowed. Network use, package installation, Git staging/commits and reading outside the repository except declared tools/profile are prohibited. Every shell sources the same declared profile. The final structured response reports a concise change summary and commands actually run; it does not supply a grade or usage claim.

2.3 p01: termcolor/termcolor

Repository: https://github.com/termcolor/termcolor. Pinned commit: c7d0b8e8a42f417ec0c07fd846138b0a19b24d36. Local repository ID: pr01. The retained tree has 28 files and 54,049 bytes. Original license text: `COPYING.txt`.

Archive SHA-256: 381e51325937b9f9af44f50b43afec28c7bb42d0852d9494b726fe76eec2d148.

> Add exported termcolor.colored_lines with colored's arguments. Color each nonempty line independently, preserving CRLF, CR, LF and empty lines exactly; treat other separators as content. Honor existing color flags, support iterable attrs, and leave colored unchanged. Add regression tests.

Original public baseline: 99 passing behavioral checks; 0 original declaration-check suites. Frozen B/C readiness: Source preload.

- pytest on tests/test_termcolor.py (89 expected behavioral checks).
- pytest on tests/test_benchmarks.py with the explicit benchmark plugin and disabled benchmarking (10 expected behavioral checks).

2.4 p02: mbr/asciitree

Repository: https://github.com/mbr/asciitree. Pinned commit: 66a66b2c3323c77aa52f927dbbd818e396b8e457. Local repository ID: pr02. The retained tree has 16 files and 25,287 bytes. Original license text: `LICENSE`.

Archive SHA-256: 1a9bdb63388decb1740efc646bc8907e2a721b91372533c2b0579fd6b71c3c60.

> Add exported asciitree.SortedDictTraversal with optional key and reverse arguments. Sort each node's children by their labels, applying key to labels when supplied; preserve equal-key order and caller data. Keep existing DictTraversal and root selection unchanged, and support LeftAligned rendering. Add regression tests.

Original public baseline: 2 passing behavioral checks; 0 original declaration-check suites. Frozen B/C readiness: Source preload.

- pytest on `tests` (2 expected behavioral checks).

2.5 p03: patrys/httmock

Repository: https://github.com/patrys/httmock. Pinned commit: 98269075d0c36a0e87e2b6b7ad9c6a16622aba58. Local repository ID: pr03. The retained tree has 9 files and 25,691 bytes. Original license text: `LICENSE`.

Archive SHA-256: 78f648904f6076742de7ffcd96fbfbefa5f817d28322de3ea9763b104c7a175e.

> Extend httmock.urlmatch method matching to accept a string or iterable of strings, case-insensitively. None remains unrestricted; an empty iterable never matches. Consume generators once when decorating, reject non-string members with TypeError, and preserve other URL filters and bound-method use. Add regression tests.

Original public baseline: 33 passing behavioral checks; 0 original declaration-check suites. Frozen B/C readiness: Source preload.

- pytest on `tests.py` (33 expected behavioral checks).

2.6 p04: keis/base58

Repository: https://github.com/keis/base58. Pinned commit: 2fae7065e3446e69e5d748c767011f05eee8400e. Local repository ID: pr04. The retained tree has 15 files and 23,864 bytes. Original license text: `COPYING`.

Archive SHA-256: 8d0d211e6186404a882e865576c6a998bfb949b5facb850f230f6458b88af537.

> Add keyword-only max_output_bytes to base58.b58decode and b58decode_check. None preserves behavior; a nonnegative integer bounds returned payload bytes before allocation, excluding the checksum for checked decoding. Reject booleans/nonintegers with TypeError and negative or exceeded bounds with ValueError. Preserve alphabets, autofix and leading zeros. Add regression tests.

Original public baseline: 48 passing behavioral checks; 0 original declaration-check suites. Frozen B/C readiness: Source preload.

- pytest on `test_base58.py`, `test_base45.py` with the explicit benchmark plugin and disabled benchmarking (48 expected behavioral checks).

The feature contract asks for bounds before allocation. The committed black-box oracle checks limits, outputs and error behavior, but cannot prove allocation order. Passing this rubric therefore does not establish that entire implementation-order requirement.

2.7 p05: mewwts/addict

Repository: https://github.com/mewwts/addict. Pinned commit: 75284f9593dfb929cadd900aff9e35e7c7aec54b. Local repository ID: pr05. The retained tree has 10 files and 34,317 bytes. Original license text: `LICENSE`.

Archive SHA-256: d425fb9a924867b1071d85e10bdac19172215460cc5bc8417d6f24c9b8a6cf61.

> Make addict.Dict.freeze and unfreeze traverse nested lists and tuples as well as Dict values, including cyclic/shared containers, without converting containers or looping. Preserve existing-key updates, reject new keys while frozen, and restore auto-vivification when unfrozen. Add regression tests.

Original public baseline: 128 passing behavioral checks; 0 original declaration-check suites. Frozen B/C readiness: Source preload.

- pytest on `test_addict.py` (128 expected behavioral checks).

2.8 p06: tartley/colorama

Repository: https://github.com/tartley/colorama. Pinned commit: 841634ed2a0da5d5ac2d867db533da8131266cb2. Local repository ID: pr06. The retained tree has 49 files and 198,802 bytes. Original license text: `LICENSE.txt`.

Archive SHA-256: ef126ce6e701c5521bfdfec6df98aa0082961a1689d2c875dc030d735bde69a9.

> Add rgb(red, green, blue, *, background=False) to colorama.ansi and export it from colorama. Return the ANSI 24-bit foreground escape by default, or background escape when requested. Accept only integer channels from 0 to 255; booleans/nonintegers raise TypeError and out-of-range values raise ValueError. Preserve existing constants and add regression tests.

Original public baseline: 38 passing behavioral checks and 14 platform skips; 0 original declaration-check suites. Frozen B/C readiness: Exact-task passthrough because candidates are ambiguous.

- pytest on `colorama/tests` (38 expected behavioral checks).

2.9 t01: jonschlinkert/kind-of

Repository: https://github.com/jonschlinkert/kind-of. Pinned commit: 3c7e8fd86094cfdb1b1bd048a39f699161da62e4. Local repository ID: r07. The retained tree has 46 files and 55,783 bytes. Original license text: `LICENSE`.

Archive SHA-256: e0dea9237fc2edfa97f8611a14f2ea8f9dad534a1fa25befe87b0cf97a6c9e16.

> Extend kindOf to distinguish async functions, async generator functions, and async generator objects as asyncfunction, asyncgeneratorfunction, and asyncgenerator. Preserve synchronous classifications and existing behavior.

Original public baseline: 36 passing behavioral checks; 0 original declaration-check suites. Frozen B/C readiness: Exact-task passthrough because candidates are ambiguous.

- Mocha on `test/` with the dot reporter (36 expected behavioral checks).

2.10 t02: sindresorhus/split-on-first

Repository: https://github.com/sindresorhus/split-on-first. Pinned commit: e426fc23a83abf43f99c3f5bc36e66d3e841906f. Local repository ID: r08. The retained tree has 13 files and 6,716 bytes. Original license text: `license`.

Archive SHA-256: 211a7d76711fa99bd26e01cfec52d89016daddec0e151563436a4dffc5db8dc4.

> Extend splitOnFirst with optional {fromEnd:true} for string separators, splitting at the last occurrence. Regular-expression separators must still split at the first match. Reject nonboolean fromEnd values and update the TypeScript declaration.

Original public baseline: 2 passing behavioral checks; 1 original declaration-check suite. Frozen B/C readiness: Exact-task passthrough because candidates are ambiguous.

- AVA with original test discovery and the external no-cache setting (2 expected behavioral checks).
- tsd on `index.test-d.ts` against `index.d.ts` through the frozen executable binding.

2.11 t03: sindresorhus/strip-final-newline

Repository: https://github.com/sindresorhus/strip-final-newline. Pinned commit: 4e80ba16dc381d4b86f5fb686cf66805d83a9c12. Local repository ID: r09. The retained tree has 13 files and 7,406 bytes. Original license text: `license`.

Archive SHA-256: 6cffe5bccdcf3acd1d97b6d7f554659cfd430ddeffcd98cedfb9b7038237c6b5.

> Add an optional {all:true} mode to stripFinalNewline that removes every trailing LF or CRLF sequence from strings and Uint8Array views. Preserve default single-newline behavior and zero-copy binary results; update the generic TypeScript declaration.

Original public baseline: 15 passing behavioral checks; 1 original declaration-check suite. Frozen B/C readiness: Exact-task passthrough because candidates are ambiguous.

- AVA with original test discovery and the external no-cache setting (15 expected behavioral checks).
- tsd on `index.test-d.ts` against `index.d.ts` through the explicit 0.31.0 legacy entry point.

2.12 t04: sindresorhus/path-key

Repository: https://github.com/sindresorhus/path-key. Pinned commit: b63661d152c4c881861361a2c03fbbbcc3aaf8e1. Local repository ID: r10. The retained tree has 13 files and 6,923 bytes. Original license text: `license`.

Archive SHA-256: 3d3ddde174859db95939057f9c69f3051681c3716d609c881f1be43813813cc4.

> Export pathKeys(options) alongside default pathKey. Return all existing PATH keys in reverse insertion priority on Windows, and only an existing literal PATH on other platforms. Never fabricate fallback entries; preserve pathKey and update declarations.

Original public baseline: 5 passing behavioral checks; 1 original declaration-check suite. Frozen B/C readiness: Exact-task passthrough because candidates are ambiguous.

- AVA with original test discovery and the external no-cache setting (5 expected behavioral checks).
- tsd on `index.test-d.ts` against `index.d.ts` through the frozen executable binding.

2.13 t05: component/escape-html

Repository: https://github.com/component/escape-html. Pinned commit: b42947eefa79efff01b3fe988c4c7e7b051ec8d8. Local repository ID: r11. The retained tree has 14 files and 17,024 bytes. Original license text: `LICENSE`.

Archive SHA-256: 615623c15cb33c081c2d77ad45fcd2ce3169e54fa01e40a46ac208a94a4b313c.

> Extend escapeHtml with an optional quoteStyle option: 'single', 'double', 'both', or 'none'. Escape only selected quotes while always escaping ampersands and angle brackets. Preserve defaults and input coercion; reject other quoteStyle values.

Original public baseline: 30 passing behavioral checks; 0 original declaration-check suites. Frozen B/C readiness: Source preload.

- Mocha on `test/` with the dot reporter (30 expected behavioral checks).

2.14 t06: sindresorhus/yocto-queue

Repository: https://github.com/sindresorhus/yocto-queue. Pinned commit: 72a8fa96a9d389765cdf2bb9c6daba8302fc375d. Local repository ID: r12. The retained tree has 13 files and 10,855 bytes. Original license text: `license`.

Archive SHA-256: 950ca6b5d09edc628c05bb3c7d149d26749f89223683f397d9f36e197c1fde1d.

> Add Queue.dequeueMany(maxCount), removing and returning up to maxCount values in FIFO order, including undefined entries. Require a nonnegative safe integer and throw RangeError otherwise. Preserve queue invariants and add the generic TypeScript declaration.

Original public baseline: 7 passing behavioral checks; 1 original declaration-check suite. Frozen B/C readiness: Source preload.

- AVA with original test discovery and the external no-cache setting (7 expected behavioral checks).
- tsd on `index.test-d.ts` against `index.d.ts` through the frozen executable binding.

3 Frozen installed-system protocol

3.1 Conditions and expected activation

A is the installed launcher in explicit off/pass-through mode. B enables v4 source discovery and compact complete selected source. C uses the same source policy and adds the locked public-command/baseline-count card from a fresh, explicitly verified receipt. The intervention changes context preparation around the same coding model. The two primary contrasts are C minus A and C minus B.

Zero-model installed readiness fixes the expected treatment for every task. p01–p05, t05 and t06 preload source. p06 and t01–t04 preserve the exact task because candidates are ambiguous. These seven active and five abstaining tasks

define descriptive readiness strata. All twelve remain in the primary intention-to-treat panel, including preparation overhead on abstentions. There are no subgroup hypothesis tests, and the active subset cannot replace the primary panel.

The same short feature request enters each launcher. Common safety/tool instructions use native developer_instructions, and project_doc_max_bytes=0 disables uncontrolled project-document injection in all arms. Added context changes the actual native prompt only as assigned. Thirty-six zero-model installed handoffs establish treatment before freeze. The bridge records actual native prompt bytes and arguments, validates their bindings, and executes the installed Codex once in the same process. The first scheduled assignment also serves as connectivity verification; no additional paid smoke session is added.

3.2 Allocation and stopping

One task in each repository receives three arms and two repeats, giving 72 scheduled workflows and 24 cells per arm. Seed 20261027 shuffles tasks and a multiset containing every arm permutation twice. The first-repeat order is:

Task block	Task	Arm order
1	t04	C → A → B
2	p05	B → C → A
3	t01	B → A → C
4	p06	C → A → B
5	p02	A → C → B
6	t05	B → C → A
7	t02	C → B → A
8	t03	A → C → B
9	p04	A → B → C
10	t06	B → A → C
11	p01	C → B → A
12	p03	A → B → C

Repeat two reverses the complete first-repeat assignment sequence, including both task-block order and within-task arm order. Assignments execute sequentially. Each receives a fresh source copy, Git baseline, local project state and native thread. This balances order under the saved design but cannot eliminate filesystem or provider-cache carryover.

An assignment is durably allocated once. There are no native retries, replacements or outcome-driven extra repeats. The next-unused-assignment controller stops on unknown whole-attempt usage, native lifecycle failure, integrity failure or exhausted admission allowance. Unrun cells remain unavailable. The planned maximum is 72 coding-agent dispatches, with a soft USD 18 allowance in uniform tariff equivalents and a USD 0.25 next-call reserve. Neither value bounds account billing or an admitted call's cost.

3.3 Time boundaries and caches

Public-source copying and Git-baseline construction are excluded from all workflow timers. Timing starts before arm-specific registration, indexing or verification, and ends after native process-group cleanup. C includes public

baseline verification and removal of only newly generated logs, build outputs and caches. Source/Git bytes and the provider manifest are checked afterward. This cleanup is an experimental control, not automatic shipping behavior. Indexing, verification, cleanup, launcher preparation and native intervals are recorded separately.

A shared absolute 90-second allowance covers setup through launcher preparation; it is not 90 seconds for each component. Every native process has a separate 300-second allowance beginning at the recorded bridge handoff. Native wall time includes final local exec setup and cleanup, not isolated remote inference. Independent grading, environment acquisition, coordinator engineering and artifact packaging are excluded. No unrelated heavy validation runs during measured execution.

The profile fixes TERM=xterm and disables supported runtime caches. Projects and run caches are fresh, while operating-system page caches, provider prefix caches, cloud load and model-backend revisions remain uncontrolled. Reported cached input is retained rather than attributed to a particular causal source.

3.4 Author controls, private inputs and grading

Separate AI authoring roles proposed tasks and constructed reference, baseline and plausibly incorrect variants; an additional agent reviewed agreement and records. This is automated role separation, not independent human replication. Across twelve tasks there are 36 expected control outcomes: baseline fails the feature, reference passes, and the incorrect variant fails while original public tests remain unchanged. Feature, public and declaration checks are finite and cannot establish exhaustive correctness.

Before native collection, all new private grader/reference/control plaintext, compiled copies and revealing logs were removed from the executor. The 3,151 committed files were packed, encrypted and authenticated with AES-GCM. A random 256-bit key remained only in the external orchestrator store. Ciphertext and hash commitments could remain locally; every private plaintext root had to be absent before each dispatch. This sequesters these oracles without claiming kernel-level isolation from arbitrary local data. Restoration occurs only after all native timings close or a durable stop, and exact committed file hashes must match before restored graders execute.

Before grading native patches, all 36 original author variants were replayed successfully through the same final frozen sandbox wrapper. TypeScript control-helper working-directory/module-ancestry diagnostics were retained; only the location of that unsealed control helper was corrected to the actual grading ancestor. The frozen private cases, feature oracles, original public checks and runtime were unchanged. These control processes made no model requests. The final wrapper had been tightened before freeze; there was no post-freeze study/grading amendment.

Independent feature and original-public grading use candidate source read-only, network disabled and only grader artifacts writable. Inherited global temporary write roots are disabled. Candidate and protected-input hashes are checked before and after grading. The strict gate requires valid native completion and whole-attempt usage, faithful treatment, intact protected inputs, valid grading infrastructure, committed feature checks and original public checks. Regression-test presence and execution remain diagnostics. A passing rubric cannot certify semantic test quality, full free-form instruction compliance or the unobservable base58 allocation order.

3.5 Accounting and inference

The primary accounting convention is [(input − cached input) × 5 + cached input × 0.20 + output × 15] / 1,000,000, in API-equivalent dollars. Cached input is part of total input; reasoning output is included in output counters. These fixed historical rates do not represent current prices, invoices, subscription charges or all provider computation. Ordinary failed attempts retain observed costs. Unknown usage is never replaced with zero.

For each primary contrast, both repeats are averaged within each task before forming the paired arm difference. A complete estimate requires all twelve task clusters and both repeated costs for both contrasted arms. Otherwise the primary total, effect and test are unavailable; known subtotals and missing identifiers remain reportable. The primary family is the two contrasts C−A and C−B, not a collection selected after results.

Complete contrasts receive pointwise 95% paired task-bootstrap intervals using 10,000 resamples, seed 20261029 and linear interpolation at (n−1)p. Each draw retains repeats and paired arms. Exact two-sided tests condition on the third arm's first-repeat position, creating three strata of four tasks with two relative orders each. Enumerating choose(4,2)^3 = 216 legal allocations and jointly swapping repeat labels respects the balanced design. Ties count. Pairwise sharp-null and no-cross-arm/provider-cache-carryover assumptions are required. Holm adjustment covers the two predeclared cost tests at familywise 0.05; missingness does not shrink that family.

Native-price/cache-write sensitivities, tokens, workflow components, readiness strata and joint-success time are secondary descriptive analyses. Command-text read/test counts are heuristic and do not trace all file I/O or provider inferences. Cheaper failed work is not evidence of greater efficiency. Matching strict-success counts does not establish population quality equivalence or formal noninferiority.

Precision planning used seven earlier complete two-repeat task clusters. Their unstable cost-difference standard deviation was 0.03441 dollars. An optimistic normal approximation suggests a detectable difference near 0.03063 dollars, roughly 25% of the earlier A mean, for twelve clusters at 80% power and first-step two-sided alpha 0.025. This is not achieved power or a guarantee for the exact test. Adaptive prototype observations are not pooled with this new installed-system panel.

4 Historical evaluation lineage

Each stage below has its own corpus, treatment, source version and measurement boundary. Some entries are local engineering or derived analyses; their observations are not new coding-agent sessions. An unsuccessful original attempt remains in its cohort even when a later repair succeeds. Figures and percentages are reported under their original definitions, with scope limits retained. The detailed history inventory binds the source report bytes.

4.1 Historical stage 1: Initial access attempt

96 planned coding assignments remained not_run; 168 real SDK calls on 24 tasks, target-file hit 16/24 (66.7%).

Scope and interpretation: No completed native A/B pair; SDK retrieval, byte estimates and 203 engineering checks are not model efficacy or billing evidence.

Original report: benchmarks/assessment.md.

4.2 Historical stage 2: Native MCP v1

96 original attempts: A 48/48 graded passes, B 47/48 plus one harness interruption; 47 matched pairs: B 2.751x total input, 1.465x Codex-plus-index time, no observed correctness gain.

Scope and interpretation: 24 synthetic tasks and correlated repeats; one interrupted B has unknown usage/time/cost; 97th supplemental B pass has no fresh comparator and is excluded. Exact call-time freshness and semantic completeness unavailable.

Original report: benchmarks/live-ab-assessment.md.

4.3 Historical stage 3: Optimized native MCP

160 attempts, A 80/80 and B 80/80 strict passes; B total input +32.82%, output +5.31%, uncached input +2.17%, Codex-plus-index time +17.88%.

Scope and interpretation: 40 small synthetic tasks with correlated repeats and success ceiling; actual cost unavailable. Two old-task development A/B pilots are separate. Complete declared bodies do not prove semantic completeness or live call-time freshness.

Original report: benchmarks/optimized-ab-assessment.md.

4.4 Historical stage 4: Known-path daily preload

80/80 strict passes on 20 tasks x2 arms x2 repeats; B total input -2.94%, output -6.15%, full time -4.76%, uncached input +27.14%; input worse in 25/40 pairs.

Scope and interpretation: Legacy suffix layout, tiny known-file synthetic tasks, no MCP or memory reuse; actual and price-equivalent cost unavailable in this study. Read-command proxies are not actual I/O.

Original report: benchmarks/daily-ab-assessment.md.

4.5 Historical stage 5: Prefix/suffix format

40/40 strict passes; compact prefix vs legacy suffix: total input -2.01%, uncached input +3.90%, full time -11.16%. The 16 actual source-format pairs have input +0.63%, uncached +1.53%; aggregate input drop comes from 4 identical-prompt controls.

Scope and interpretation: Neither arm is bare Codex; one repeat, tiny known-file tasks, no same-file reuse condition. 31.04% envelope-byte reduction is not model token reduction. Different corpus/arms prohibit treating +27.14% to +3.90% as iteration gain.

Original report: benchmarks/cache-refinement-assessment.md.

4.6 Historical stage 6: Ten-task tariff analysis

20/20 strict passes; pooled B cost/success estimate 2.28% lower; 8 source-preload tasks 4.43% higher, 2 identical-prompt controls 33.27% lower. B cheaper in 5/10 pairs.

Scope and interpretation: One synthetic repeat; controls account for apparent pooled benefit. Unknown cache-write attribution allows -15.47% to +16.99% saving scenarios, not a confidence interval; no actual invoice.

Original report: benchmarks/cost-assessment.md.

4.7 Historical stage 7: Original workflow contract

60 native attempts, 20 per A/B/C; all behavior/smoke checks passed but all 60 strict failures changed protected existing tests. Cost/success unavailable with zero strict successes.

Scope and interpretation: Shared prompt was insufficiently explicit about exact test bytes; behavior-only results cannot replace frozen strict outcomes. Per-arm attempt cost remains charged.

Original report: benchmarks/workflow-assessment.md.

4.8 Historical stage 8: Fresh workflow confirmation

60/60 strict passes on 10 fresh tasks x3 arms x2 repeats. All-task B vs A estimated cost/success -2.475%; focused 16/arm B cost -10.19% but mean full time +3.18%; C vs B focused cost +8.66%, time +9.16%.

Scope and interpretation: Small synthetic tasks; known paths/support caller selected, zero configured MCP; support promotion gate false. Focused B cache-write scenario saving -3.54% to +22.49%; no actual bill or joint cost/speed win.

Original report: benchmarks/workflow-confirmation-assessment.md.

4.9 Historical stage 9: Workflow accounting

120 original assignments (the preceding two cohorts), 40/arm with 20 strict successes/arm; all failed-attempt costs included. Estimated total A 1.2889548/B 1.2826008/C 1.2474700 USD.

Scope and interpretation: Do not count another 120 sessions or pool distinct corpora for efficacy/time. Local source-reader 55 to 15 and 1, 200 synthetic cases are separate component evidence.

Original report: benchmarks/efficiency-assessment.md.

4.10 Historical stage 10: Local efficiency work

600-file graph mean wall -11.75%, unchanged secure reads 600; 40-file/20MiB scan Python allocation peak -92.56%, reads 40; virtual scheduler connections 120 to 12; dashboard rows 500 to 20; SQLite instruction reductions with larger DB and insert 44 to 55 instructions.

Scope and interpretation: Zero new native efficacy requests. tracemalloc is not RSS; graph old branch emulation, scan/graph overlap, virtual ticks and SQLite VM instructions are not production task latency/throughput/model tokens. Hosted SaaS readiness unproven.

Original report: benchmarks/saas-efficiency-assessment.md.

4.11 Historical stage 11: Warm-study access failure

216 planned; 1 original auth failure before a completed model turn; 215 unrun; zero warm stages/quotes.

Scope and interpretation: Unknown tokens/cost are not zero; auth-failure seconds are not coding latency. Capability checks establish reuse only inside same active Memglide session.

Original report: benchmarks/warm-study-assessment.md.

4.12 Historical stage 12: Partial warm study

80 completed strict passes; 81st user-interrupted with unknown usage; 135 unrun. Post-hoc first 72 matched runs:24 tasks/arm in 8 sequences; B vs A cost -9.055%, instrumented mean time -6.892%; warm C vs B cost +12.346%, time +3.396%.

Scope and interpretation: Post-hoc partial first repeat, unequal full observed task mixes, small synthetic roots, uncontrolled caches. C full attempted cost unknown; same active Memglide session, not durable cross-session memory; cache-write scenarios can reverse ranking.

Original report: benchmarks/warm-80-analysis.md.

4.13 Historical stage 13: Automatic v1

12 path-free tasks: C delivers0/12, known-path B 9/12; 24 exact-source/passthrough checks pass. Local setup+prep C 1.048s vs B 0.728s.72 planned native assignments not run.

Scope and interpretation: C zero added bytes means no source delivered, not savings; fixed B-before-C one repeat and tiny mutation tasks. Grader baseline/reference/partial-fix controls are author controls, not model-generated repairs.

Original report: benchmarks/automatic-assessment.md.

4.14 Historical stage 14: Router v2

72 offline observations: B 24/24 target scope, V1 zero, C 2/24 preloads but wrong type-only file, target0/24; fixed gate failed, 72 authorized native slots unrun. C cold mean 2.098610s vs B 1.216337s (+72.54%).

Scope and interpretation: 12 mutation tasks/4 small roots, 2 correlated repeats; B human discovery effort omitted; easier 18-observation Python synthetic profile had 18 target deliveries but did not predict held-out. Source-fidelity72/72 is not relevance or coding success.

Original report: benchmarks/router-v2-assessment.md.

4.15 Historical stage 15: Local source refinement

72 timing+6 diagnostic workers, 468 preparations, 24 fallback controls preserve exact source/prompt; compilation 6 to 3 typical/2 to 1 dense. Typical warm mean -2.08% but all primary mean intervals include zero; dense warm median improves while mean+10.14%.

Scope and interpretation: No model or real indexing; synthetic approvals. Five calls form one worker mean, not 5 independent observations. Outliers retained; 77.9% no-ignore apparent mean gain is no-effect control confounding. RSS whole worker, API reads not actual I/O.

Original report: benchmarks/source-only-refinement-assessment.md.

4.16 Historical stage 16: Router v3

96 offline observations, 12 tasks x4 arms x2 repeats; positives C 6/16 (3/8 unique) vs V2 4/16, V1 zero, known-path B 16/16; all arms8/8 negative bypasses. Positive C prep mean+7.00% vsV2, setup+prep+1.12%, intervals cross zero.

Scope and interpretation: 8 positive tasks share7 bundles/4 roots including2 related unjs roots; source-scope proxy only. Negative bypass never offsets positive miss; no token/cost/coding-success evidence.

Original report: benchmarks/router-v3-assessment.md.

4.17 Historical stage 17: Router v4 checkpoint

No authored fresh held-out at checkpoint; 27 fake-native runner controls are harness checks; one real access probe timed out with proxy403 and no completed model turn.

Scope and interpretation: Checkpoint status is historical; later native studies supersede pending wording without overwriting it. No measured efficacy, and unknown usage is not zero.

Original report: benchmarks/router-v4-checkpoint.md.

4.18 Historical stage 18: Router v4 blocked

6/72 scheduled attempts (2/arm) runtime blocked shell/patch; 66 unrun, 0 strict passes; measured tariff point 0.1881096 USD, possible-write 0.2259976.

Scope and interpretation: No patch and no valid coding-performance comparison. Offline gate not met and explicitly waived before corpus; two failed earlier access preflights have unknown costs. Later successful runtime fix does not replace these 6.

Original report: benchmarks/router-v4-native-blocked-assessment.md.

4.19 Historical stage 19: Router v4 recovery

72/72 attempts produce patches: strict A 0/24 B 1/24 C 0/24; raw behavior22/24 each; 68 existing-test byte violations. C input -2.06%, execution+prep -8.42%, mean cluster intervals cross zero; C preload 8/24.

Scope and interpretation: Exposed reused 12 tasks, offline eligibility waived, ambiguous byte-preservation prompt, native tool discovery incomplete. Raw behavior is diagnostic and not substituted for strict.312857-token aggregate creates unknown point price for coding-0044.

Original report: benchmarks/router-v4-native-recovery-assessment.md.

4.20 Historical stage 20: Router v4 confirmation

72 attempts: strict A 22/24 B 20/24 C 23/24; 6 ordinary failures+1 A timeout; C source 0/24 and every C prompt matches A; B preload 16/24.11 cost-complete task clusters: C-A point+0.00207484 USD/attempt, CI[-0.00506235,+0.00811680].

Scope and interpretation: C abstention cannot establish preload mechanism efficacy.12 authored tasks/6 small roots, partial upstream snapshots. Timeout unknown cost makes whole 72 total and A cost/success unavailable. Timing 12clusters mean-3.911s CI crosses zero; median+2.687s slower. Original 16-row prefix imported unchanged once.

Original report: benchmarks/router-v4-native-confirmation-assessment.md.

4.21 Historical stage 21: Active v4 pilot

32 native attempts on 8 fresh task clusters x2 arms x2 repeats; A 16/16 C 15/16; C source 16/16. C attempt tariff cost-2.88%, cold time-4.74%, cost per strict pass+3.59%; mean cost/time CIs cross zero.

Scope and interpretation: Conditional eligibility; 17 candidate repositories with query screening, declines and pre-paid wording revisions, not blind activation prevalence. Failure n04 malformed1s, preserved. Historical incomplete charges stay unknown.

Original report: benchmarks/router-v4-active-pilot-assessment.md.

4.22 Historical stage 22: Diagnostic repair

One diagnostic native session repairs known failed n04 patch; 115 original+19 boundary checks pass. Charging failure+repair makes C workflow cost+1.69% and time+1.47% vs original A; pilot staysA16/16 C 15/16. Local 100-file early-fallback mean 48.60ms to 0.187ms, 10 pairs.

Scope and interpretation: Feedback-informed asymmetric repair is development, not new independent A/C confirmation; local fallback result is not Codex speed/tokens. Original 32 outcomes not rescored.

Original report: benchmarks/router-v4-repair-assessment.md.

4.23 Historical stage 23: Adaptive v5 and fresh comparison

30 candidate rounds x3 development tasks+3 source-only reference =93 native sessions; 32 fresh A/C attempts; 125 scheduled terminal records, 124 valid native completions. Fresh strict 15/16 each. Primary 7 complete task clusters/14 pairs:18.7260405% lower C uniform-tariff cost; mean delta-0.0226900 USD, 95%[-0.0469470,+0.0010488].

Scope and interpretation: Winner selected adaptively on 3 seen tasks; 8 screened small preload-eligible fresh roots, 2 repeats; C capacity failure v5-final-0031 unknown usage/cost, no retry. Primary CI crosses zero; secondary tariff CI excludes zero does not supersede primary. Equal strict counts do not establish quality equivalence. Not source-only ablation; verified public-command workflow package.

Original report: benchmarks/router-v5-search-assessment.md.

Historical conversions that are not supported include equating duplicate MCP transport JSON with duplicate model input, envelope-byte reductions with token savings, Python allocation peaks with RSS, or command text with complete filesystem traces. Source hashes and complete selected files establish a narrower property than complete task understanding. Equal finite pass counts do not establish quality equivalence.

5 The thirty-round v5 development search

5.1 Rules and interpretation

The v5 development screen used three fixed tasks, three initial reference sessions and 30 candidates with one native session per task. Each candidate was frozen before its round. Ninety candidate sessions plus three references give 93 development sessions. Reference and candidate rounds used the same three-worker submission policy; recorded submission order did not guarantee native start order. Cached provider state and concurrent contention remained uncontrolled. Final v5 evaluation ran sequentially after the winner lock.

Candidate changes were portable prompt/preparation/workflow changes. They did not retrain the coding model. The candidate had to preserve task text and source provenance and could not encode task IDs, private expectations or reference solutions. A candidate branched from the current incumbent; rejected versions and their measurements were retained rather than destructively reset.

Promotion first required valid complete native/treatment/usage records and no protected-input or infrastructure failure. A candidate could not lose a task passed by its incumbent. Before the incumbent reached 3/3, promotion required strict pass-vector dominance. At 3/3, a candidate had to keep 3/3 and improve aggregate cost or time by at least 5%, while the counterpart worsened by at most 5%. Each task passed by both versions also had to remain within 130% of its incumbent cost and time. Exact thresholds were admitted; ties kept the incumbent. These fixed engineering screens over three observations were not significance tests. Thirty adaptive looks create selection bias and winner's-curse risk.

The entries below reproduce each saved hypothesis, effective configuration change, strict outcome, both tariff conventions, summed preparation-plus-native time, parent and recorded decision. Every aggregate includes all three attempts, including failures. Times are sums of the three preparation/native components, not the wall-clock duration of a parallel round. Promotion used the separately retained frozen controller timing measure. The original strict grader is unchanged; the later compiler-alignment diagnostic does not rescore these primary rounds.

5.2 Reference round 0: v00

Hypothesis: Current production automatic source with the common baseline instructions

Configuration: the then-current production automatic source and common baseline instructions; no candidate change.

Outcome: 2/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4047176 dollars; native point-price equivalent: 0.1916088 dollars. Summed preparation plus native time: 151.546 seconds.

Decision: Retained as the initial reference. Incumbent afterward: `v00`.

5.3 Candidate round 1: v01

Hypothesis: Verified public commands remove exploratory setup commands.

Effective change: command_card=commands. Parent: `v00`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.5471604 dollars; native point-price equivalent: 0.2499032 dollars. Summed preparation plus native time: 152.229 seconds.

Decision: Promoted for strict correctness dominance. Incumbent afterward: `v01`.

5.4 Candidate round 2: v02

Hypothesis: Baseline test counts prevent zero-discovery false confidence.

Effective change: command_card=commands_counts. Parent: `v01`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.3970016 dollars; native point-price equivalent: 0.1866728 dollars. Summed preparation plus native time: 132.879 seconds.

Decision: Promoted under the fixed cost/time screen. Incumbent afterward: `v02`.

5.5 Candidate round 3: v03

Hypothesis: Readiness and unavailable-check facts avoid dependency/config retries.

Effective change: command_card=readiness. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.5032054 dollars; native point-price equivalent: 0.2346772 dollars. Summed preparation plus native time: 150.208 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.6 Candidate round 4: v04

Hypothesis: One complete related public test clarifies retained behavior without discovery.

Effective change: public_tests=one_import_linked. Parent: `v02`.

Outcome: 2/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4461438 dollars; native point-price equivalent: 0.2100484 dollars. Summed preparation plus native time: 159.906 seconds.

Decision: Rejected because it lost a task passed by the incumbent. Incumbent afterward: `v02`.

5.7 Candidate round 5: v05

Hypothesis: A second complete related public test exposes more compatibility behavior.

Effective change: public_tests=two_import_linked. Parent: `v02`.

Outcome: 2/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4422368 dollars; native point-price equivalent: 0.2078944 dollars. Summed preparation plus native time: 146.099 seconds.

Decision: Rejected because it lost a task passed by the incumbent. Incumbent afterward: `v02`.

5.8 Candidate round 6: v06

Hypothesis: Public tests and readiness facts jointly reduce inspection and setup work.

Effective change: command_card=readiness, public_tests=two_import_linked. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4784382 dollars; native point-price equivalent: 0.2196036 dollars. Summed preparation plus native time: 139.667 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.9 Candidate round 7: v07

Hypothesis: An executable public feature expectation constrains implementation.

Effective change: regression_stage=feature_first. Parent: `v02`.

Outcome: 2/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4237386 dollars; native point-price equivalent: 0.1992868 dollars. Summed preparation plus native time: 152.417 seconds.

Decision: Rejected because it lost a task passed by the incumbent. Incumbent afterward: `v02`.

5.10 Candidate round 8: v08

Hypothesis: Testing both new behavior and preserved boundaries reduces regressions.

Effective change: regression_stage=feature_boundary_first. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4199908 dollars; native point-price equivalent: 0.2020104 dollars. Summed preparation plus native time: 186.140 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.11 Candidate round 9: v09

Hypothesis: A short contract ledger makes test expectations independent of the patch.

Effective change: regression_stage=contract_then_tests. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.5260136 dollars; native point-price equivalent: 0.2382728 dollars. Summed preparation plus native time: 143.742 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.12 Candidate round 10: v10

Hypothesis: A separate diff review finds changed-branch mistakes.

Effective change: compatibility_review=changed_diff. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.5420548 dollars; native point-price equivalent: 0.2521064 dollars. Summed preparation plus native time: 178.913 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.13 Candidate round 11: v11

Hypothesis: A dedicated retained-API review catches untested old behavior.

Effective change: compatibility_review=preserved_api. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.5613492 dollars; native point-price equivalent: 0.2576256 dollars. Summed preparation plus native time: 161.221 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.14 Candidate round 12: v12

Hypothesis: A lifecycle/fallback review catches mutation and asynchronous edge cases.

Effective change: compatibility_review=state_lifecycle. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.5342374 dollars; native point-price equivalent: 0.2512932 dollars. Summed preparation plus native time: 188.855 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.15 Candidate round 13: v13

Hypothesis: Summarizing successful checks reduces tool-output context.

Effective change: check_output=success_summary. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.519543 dollars; native point-price equivalent: 0.238814 dollars. Summed preparation plus native time: 154.433 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.16 Candidate round 14: v14

Hypothesis: Counts and complete relevant failure diagnostics reduce output without false passes.

Effective change: check_output=counts_and_failures. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.6026306 dollars; native point-price equivalent: 0.2740788 dollars. Summed preparation plus native time: 161.689 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.17 Candidate round 15: v15

Hypothesis: A structured receipt reduces repeated status parsing and long logs.

Effective change: check_output=structured. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.5032658 dollars; native point-price equivalent: 0.2373484 dollars. Summed preparation plus native time: 180.802 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.18 Candidate round 16: v16

Hypothesis: Targeted checks during editing reduce repeated full-suite work.

Effective change: verification=targeted_then_full. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4506736 dollars; native point-price equivalent: 0.2145088 dollars. Summed preparation plus native time: 172.311 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.19 Candidate round 17: v17

Hypothesis: A targeted baseline run reveals setup problems before code changes.

Effective change: verification=baseline_targeted_then_full. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.5471944 dollars; native point-price equivalent: 0.2564632 dollars. Summed preparation plus native time: 187.634 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.20 Candidate round 18: v18

Hypothesis: A full baseline separates preexisting failures from new regressions.

Effective change: verification=baseline_full_then_full. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.399274 dollars; native point-price equivalent: 0.187932 dollars. Summed preparation plus native time: 148.519 seconds.

Decision: Rejected because it did not meet the fixed practical-improvement thresholds. Incumbent afterward: `v02`.

5.21 Candidate round 19: v19

Hypothesis: Using readiness diagnostics first avoids repeated setup debugging.

Effective change: diagnostics=tool_error_first. Parent: `v02`.

Outcome: 2/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4708998 dollars; native point-price equivalent: 0.2153484 dollars. Summed preparation plus native time: 135.433 seconds.

Decision: Rejected because it lost a task passed by the incumbent. Incumbent afterward: `v02`.

5.22 Candidate round 20: v20

Hypothesis: Failure-local inspection reduces broad rereads and repeated passing checks.

Effective change: diagnostics=failure_localized. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4253618 dollars; native point-price equivalent: 0.1987844 dollars. Summed preparation plus native time: 147.746 seconds.

Decision: Rejected because it did not meet the fixed practical-improvement thresholds. Incumbent afterward: `v02`.

5.23 Candidate round 21: v21

Hypothesis: Batching independent inspections reduces tool round trips without a hard call limit.

Effective change: diagnostics=batch_readonly. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.6204032 dollars; native point-price equivalent: 0.2822176 dollars. Summed preparation plus native time: 165.744 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.24 Candidate round 22: v22

Hypothesis: Raw complete blocks avoid JSON escape syntax while retaining exact content.

Effective change: layout=v5_raw_task_last. Parent: `v02`.

Outcome: 2/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4005532 dollars; native point-price equivalent: 0.1867416 dollars. Summed preparation plus native time: 136.111 seconds.

Decision: Rejected because it lost a task passed by the incumbent. Incumbent afterward: `v02`.

5.25 Candidate round 23: v23

Hypothesis: An anchor/helper manifest makes the dependency bundle easier to navigate.

Effective change: layout=v5_manifest_anchor_first. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4054448 dollars; native point-price equivalent: 0.1961344 dollars. Summed preparation plus native time: 168.149 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.26 Candidate round 24: v24

Hypothesis: Task-first framing changes attention without changing task or source bytes.

Effective change: layout=v5_task_first_raw. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.3875766 dollars; native point-price equivalent: 0.1860068 dollars. Summed preparation plus native time: 161.370 seconds.

Decision: Rejected because it did not meet the fixed practical-improvement thresholds. Incumbent afterward: `v02`.

5.27 Candidate round 25: v25

Hypothesis: Comparing public baseline examples detects unlisted compatibility changes.

Effective change: differential=baseline_examples. Parent: `v02`.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4006566 dollars; native point-price equivalent: 0.1986068 dollars. Summed preparation plus native time: 204.223 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: `v02`.

5.28 Candidate round 26: v26

Hypothesis: Baseline error/default comparisons catch preserved negative contracts.

Effective change: differential=baseline_errors. Parent: `v02`.

Outcome: 2/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.5451874 dollars; native point-price equivalent: 0.2607692 dollars. Summed preparation plus native time: 208.256 seconds.

Decision: Rejected because it lost a task passed by the incumbent. Incumbent afterward: `v02`.

5.29 Candidate round 27: v27

Hypothesis: Short baseline call sequences expose fallback and state regressions.

Effective change: differential=baseline_sequences. Parent: v02.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.5411756 dollars; native point-price equivalent: 0.2596328 dollars. Summed preparation plus native time: 223.631 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: v02.

5.30 Candidate round 28: v28

Hypothesis: Boundary-first tests plus compact targeted verification improve quality per attempt cost.

Effective change: check_output=counts_and_failures, command_card=readiness, regression_stage=feature_boundary_first, verification=targeted_then_full. Parent: v02.

Outcome: 3/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.478787 dollars; native point-price equivalent: 0.22709 dollars. Summed preparation plus native time: 171.868 seconds.

Decision: Rejected because at least one both-passing task exceeded the 30% cost/time regression guard. Incumbent afterward: v02.

5.31 Candidate round 29: v29

Hypothesis: One public test plus API review and structured baseline validation balances context and correctness.

Effective change: check_output=structured, compatibility_review=preserved_api, public_tests=one_import_linked, verification=baseline_targeted_then_full. Parent: v02.

Outcome: 2/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.4193652 dollars; native point-price equivalent: 0.2043616 dollars. Summed preparation plus native time: 190.367 seconds.

Decision: Rejected because it lost a task passed by the incumbent. Incumbent afterward: v02.

5.32 Candidate round 30: v30

Hypothesis: Complete raw context plus baseline boundary comparisons and compact checks combine complementary mechanisms.

Effective change: check_output=counts_and_failures, differential=baseline_errors, layout=v5_raw_task_last, verification=targeted_then_full. Parent: v02.

Outcome: 2/3 strict passes in 3 native sessions. Uniform tariff equivalent: 0.5410302 dollars; native point-price equivalent: 0.2624196 dollars. Summed preparation plus native time: 240.056 seconds.

Decision: Rejected because it lost a task passed by the incumbent. Incumbent afterward: v02.

5.33 Selected recipe and development cost

Rounds 1 and 2 were promoted; the remaining 28 candidates were rejected. The locked v02 recipe combines verified public commands and observed baseline counts with ordinary regression/review instructions, full check output, final verification, compact v4 layout, no public-test-body addition and no differential baseline comparison. Its full configuration records command_card=commands_counts, public_tests=none, regression_stage=ordinary, compatibil

ity_review=ordinary, check_output=full, verification=full_final, diagnostics=ordinary, layout=v4_compact and differential=none.

The 93 development sessions contain 84 strict passes and a known uniform tariff subtotal of 14.8853110 dollars. The native point-price subtotal is 6.9882620 dollars. Rejected hypotheses and failed attempts are included. Coordinator assistance, engineering time and account charges are not valued by these figures. A selected development recipe is not a validated efficiency claim.

The separate eight-task v5 comparison retained both 15/16 strict counts and its one assisted provider-capacity failure with unknown usage. Its complete-case primary cost interval crossed zero. A secondary native-price interval and a later compiler-alignment diagnostic remain secondary. They do not replace the original strict scores or justify pooling v5 with the installed-system panel.

6 Source revision and environment

6.1 Pre-study source-role correction

Public readiness found that root tests.py could compete with production and legacy demo material could enter production guards. The two-line shared classifier change adds exact demo/demos directory components and exact test.py/tests.py basenames to existing nonproduction exclusions. It does not branch on repository names or task terms, alter scores or thresholds, enlarge budgets, edit fixtures or change private expectations.

Seven new regression cases failed before the correction while seven conservative controls passed. Afterward, all fourteen new cases and all 214 focused v1–v4/query/launcher checks passed; Ruff and mypy passed. Similarly named production modules continue to compete, while unsupported production files and production parse failures can still force abstention. The classifier is shared by explicitly invoked v1–v4 policies. Old results remain bound to old source snapshots and are not rescored.

The source before the correction hashes to fe19959e2d29b9461d674fcfa9c549b5da7064ff160739201e4ed21d5bd99546; the new auto-preload source hashes to e2eaa00847838af3c99dc93683bceff55b7284904f5b7c5cf9a507de8ab37a82. The patch hashes to 844b525863bcdbb0d2880c1a9f6631b90ee4cae11c8a81b1c81d6edb67ef7e0a. The existing early-query preflight optimization and selected workflow modules were preserved. This was task-informed public-readiness engineering before coding outcomes, not a blind test of an untouched selector.

6.2 Runtime and tool profile

The measured environment was a managed Linux x86_64 container, kernel 6.18.55, with controller Python 3.12.14 and Node 24.19.0. Five logical CPUs were visible; the cgroup CPU quota was 400000/100000, equivalent to four CPU-seconds per second, and the memory limit was 34,359,738,368 bytes. Shared host and provider load were not controlled. The coding model alias was gpt-6.1-sol with low effort and default tier, through Codex CLI 0.159.2. Alias identity does not pin the provider's underlying model revision.

The additive frozen profile exposes the earlier declared Python/Node tools, the isolated v6 node_modules tree and Python extensions. Pinned tools include pytest 9.0.2, Mocha 10.3.0, AVA 6.4.1, TypeScript 5.9.3, tsd 0.33.0 and the explicit tsd-legacy 0.31.0 compatibility entry point. The new Node package manifest also pins uvu 0.3.3, esm 3.2.25 and Node type declarations 24.3.0; installed availability does not mean a tool was used by an eligible task. Python extensions are PyHamcrest 2.1.0, pytest-benchmark 5.2.3 and py-cpuinfo 9.0.0. Exact resolved dependencies, executable bindings and symlink targets are retained in lockfiles and the input freeze. No package installation occurs inside a native assignment.

Before collection, product validation passed frozen dependency sync, 2,474 Python tests, Ruff, mypy, frontend installation/build, 24 frontend tests and frontend bundling. These are implementation checks, not additional coding-model successes. A first validation attempt encountered a read-only default uv cache; the retained successful attempt used an explicit workspace cache. No test, source fixture or benchmark outcome was changed to resolve that environment issue.

6.3 Freeze commitments

The input freeze binds 23,485 files, 55 symlink targets, 36 production Python paths and six previous archive hashes. Its operational record is local rather than a public preregistration. The following SHA-256 values identify the principal committed inputs; full file-level bindings remain machine-readable:

- Infrastructure/source/tool freeze: c6ba872dfb29ab70569af91c213e57785a13871b22208cbcfcb87993ffd34001.
- Prospective protocol: 9a7c23c65d04af3da24e7eb6c02c012d2d483f7993407cf831fe57b00833b244.
- 72-assignment schedule: 407695021e5af78e1d7b04170cff13846428db1b27149e3244d513cfd1349f74.
- Combined public tasks and protection metadata: bc8a0aa5db5351381e7355e0a832118d8f05cf1cf1d2128d7c76678f6327ea4b.
- Expected installed treatments: 890b2362d82aab6ef47a60ba58696f5af25404c8d609e3e621e21e4336e78d8f.
- Private grading invocation manifest: 186248c69f9371ef08a5144f8b367cb50feab71b5bbc5fbcd654c658bdca6461.
- Private-isolation commitment: c6bd646f308e8344d4803cc8d1978fd7f40f8940a26c54db8c6843c897b940ce.
- Common developer contract: 948578f133e46545e62c0615576f0268162019ca4e8c967d798859b8e6c38b81.
- Common additive profile: bce5de4c83b17672770a6a275f2999ce8ad05bf88efb534b9c7ceea4e217cb3a.
- Native response schema: c61f58ddd76140be76c3672d206dd37d704494e850150fc2a053dd896ec37936.
- Isolated v6 Node dependency lock: cda9f270b591326b833a799d08dd9731171cb858aacd2a0dc9da2fef00c8bcdc.

The private bundle's pre-encryption hash is c34daffad040793ef2a6b49ce68a9d54318089a60e8e95c04d9eefec5207df98; the authenticated ciphertext hash is 68741b1c6a86f6ef1f4b6c11d9cddc119e0e01097ae19d1c995be6246443b758. The file commitment records all 3,151 plaintext hashes. The random experiment-only decryption key was released to an append-only artifact record after all 72 native assignments had durably closed and exact restoration was verified. This enables authenticated ciphertext-to-plaintext verification. It is unrelated to account credentials; runtime authentication state is not distributed. The original isolation metadata remains unchanged.

The final frozen sandbox's postcollection author-control receipts are bound by fa3f226891d6e93dc2a574113c79bc126f721d2b5cd827a8bdd400fdf19d8130 for Python and 97c4b817901d1d08701ce8fabf0c8d89b5e2f39feb1f8d873942d27d02d335ae for JavaScript/TypeScript. They validate the grading route with known author variants, not new coding-model predictions.

The base checkout commit is 0b8e3586554a5002b6d49ded21d64b923a27a99c. The evaluated revision also includes the retained working-tree implementation and source-role patch, so checking out that commit or the later moving main branch alone does not reproduce the study. Exact source bytes and committed hash bindings identify the evaluation version.

7 Reproducibility and artifact interpretation

7.1 What can be replayed

The machine-readable record preserves task/command manifests, complete public fixtures and acquisition hashes, expected handoffs, installed runtime sources, control commitments, actual request/response captures, grades, accounting and analysis rules. Replay begins by verifying manifest hashes and separating historical snapshots from the new installed-system revision. The original working layout uses /workspace/Memglide; saved absolute executable paths and ancestor-based Node resolution are part of that environment. A relocated reproduction requires an explicit documented rebinding and fresh verification, not a claim of byte-identical frozen execution.

Analysis and grading of retained outputs do not require new coding-model requests. A new native replication requires its own authenticated supported Codex connection, the declared runtime/profile and a new prospectively frozen schedule/state. It must not overwrite original slots or treat provider, backend, account or cache differences as controlled. Credentials and account configuration are not scientific evidence and are not distributed.

The controller exposes separate `prepare-public`, `preflight`, `seal`, `verify`, `run` and `grade` phases. Public preparation combines task/protection metadata; readiness uses a separate zero-model executable; sealing binds inputs; `verify` checks that commitment. `run` allocates only unused assignments and honors stopping rules. `grade` requires native closure and exact private restoration. The bridge captures actual installed handoff rather than inferring treatment from the outer launcher's original stdin. The final analysis keeps unavailable cells explicit and applies the predeclared paired design.

For product reconstruction, use the retained source revision with frozen uv and frontend locks. The documented checks are `uv sync --frozen`, `uv run pytest`, `uv run ruff check .` and `uv run mypy src`; in frontend, run `npm ci`, `npm run build` and `npm test -- --run`, then bundle through `uv run python scripts/bundle_frontend.py` at the repository root. Workspace runtime/cache directories isolate setup state. Independent transport/frontend integration claims require their relevant integration checks in addition to these general commands.

The original six earlier evidence ZIPs are immutable. The v5 search archive is identified by fcf9609f8b751ed8b0e4d4 51068a3da46040f68a10a768187ac93dd76eec687d. Its collector-recovery amendment, unknown provider usage, original strict scores and supplemental compiler diagnostics remain separate records. The history inventory and per-cohort manifests identify earlier artifacts; their reported totals must not be added as if derived rows were new sessions.

7.2 Manuscript production and external sources

The paper and this supplement are Markdown sources with a shared bibliography and an acmart-based LaTeX build. `build.py` runs Pandoc, pdflatex and BibTeX, then records output hashes. No conference name, publisher DOI, acceptance date, affiliation or ORCID is invented. The named author is Sree Charan Reddy Kailasam, with the supplied email srcharan014@gmail.com.

The official acmart source commit is cb4bea05438ac81de82df97e8b15326df5e602b3. Fresh downloads from the original repository matched the existing installer, bibliography style, class source and previously saved pinned source byte for byte. The copied verification receipt is supplement/acmart-source-pin-verification.json. Its exact source identities are:

- acmart.ins: fa7daaa9375775ce699f33f909667bdb83c0408fd8d67606926df86c346c4044.
- ACM-Reference-Format.bst: 4342d27456645567ef1a027f6474e894715bc31d468f040142fa54540440ad04.
- acmart.dtx: 8780f6b7763852a290b0fcbc5d53a7209e324569a082aaa95ade79d2e7f7db8f.

The font setup uses isolated hash-verified Debian packages. The class source is an official development revision; production-template and submission requirements still depend on the selected venue. A successful named-preprint build is not certification of a future journal's or conference's current rules. The official class manual/sample guidance was read, and publisher-maintained ACL metadata/abstracts and identified author repositories support the bibliography. Complete article retrieval is not claimed where only abstracts, metadata or project READMEs were available.

Wikipedia's requested "Signs of AI writing" essay was not read because its domain remained blocked by the managed environment policy. No network bypass was attempted. The publication/editorial notes distinguish this missing source from actual readings of detector-bias and detector-reliability research repositories. Detector outputs do not certify authorship, and no detector-proof or unassisted-human-writing claim is made.

7.3 Licenses, privacy and author responsibility

Memglide's project license is MIT. Third-party repositories, test fixtures, dependencies, fonts and class sources retain their own licenses and copyright notices. The corpus is not relicensed wholesale under the manuscript's terms. The retained httmock notice identifies Apache License 2.0; Colorama retains its BSD-style notice; the other retained source notices are preserved in their original files. Artifact reuse must follow those notices. Complete restrictive external research READMEs, illustrations or code are not needed for a citation and should not be bundled automatically. In particular, the official detector-reliability repository contains copying/distribution restrictions.

The corpus uses public repository material. Native source context is still transmitted to an external coding model. Local index/state placement does not make inference local, and sensitive-content filtering is heuristic. Hashes bind checked files, policy and declared tools rather than all dependencies, undeclared environment settings or semantic safety. Public-command execution requires authorization; its time/output limits are not an operating-system sandbox. The stronger read-only grading boundary is an evaluation control.

OpenAI Codex assistants supported implementation, task/control generation, experiment orchestration, analysis code, source checks and manuscript drafting. Separate agents reviewed records; this does not constitute independent human replication. No AI tool is listed as an author. Verification of final claims, citations, records and text, and responsibility for any submission, rest with Sree Charan Reddy Kailasam. No completed human review or publication acceptance is asserted on the author's behalf.

References

[1] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=VTF8yNQM66